Design and Implementation of an SSA-Based RISC-V Compiler with Comparative Register Allocation

Shaun Alan Joseph	Rishi Sharma	Komal	Aswin Parames
2023BCD0006	2023BCD0023	2023BCD0011	2023BCD0024

Guided by: Dr. Nandini J Warrier

Dept. of Computer Science and Engineering

Indian Institute of Information Technology, Kottayam

B.Tech Project · Phase 1 · Review 1 · 9 September 2026

Indian Institute of Information Technology Kottayam

The last big decision a compiler makes

- A program uses unlimited named values. A CPU has few **registers** — 32 on RISC-V, 16 on x86-64
- Values that don't fit are **spilled** to memory. A spilled value costs a load and a store on *every* access — inside a loop, every iteration
- One of the highest-impact passes in a compiler backend
- **Chaitin (1981):** build an **interference graph** — one node per value, an edge between values alive at the same time — then colour it with k colours, where k = number of registers

Background: SSA Form

Static Single Assignment (SSA)

- Every value assigned **exactly once**. Reassignment creates a new value: `x1`, `x2`, `x3`
- Where control flow merges, a **φ-function** picks whichever value arrived: `x3 = φ(x1, x2)`
- Used by LLVM, GCC, HotSpot, V8 — the normal form of modern compilers

Why SSA changes the problem

- In general interference graphs can be *any* shape, and colouring an arbitrary graph is NP-hard — the textbook reason for heuristics
- Interference graphs of SSA programs are always **chordal**
- Chordal graphs colour **optimally, in linear time**
- Established by Hack & Goos and by Pereira & Palsberg, both 2005

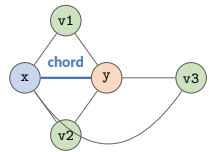
(a) C source

```
int x = f(), y = g();
int v;
if (c)  v = x + y;
else    v = x - y;
return v + x + y;
```

(b) Same code in SSA

```
x = f()
y = g()
v1 = x + y   (if c)
v2 = x - y   (else)
v3 = φ(v1, v2)
r = v3 + x + y
```

(c) Interference graph



4-cycle v1–x–v2–y has chord x–y
⇒ **chordal** ⇒ 3 colours, optimal

Literature Survey

Two lines of work

Indian Institute of Information Technology Kottayam

The classical line — all heuristics, no optimality guarantee

Work	Year	Contribution
Chaitin et al.	1981	Interference-graph formulation; showed the problem is NP-hard in general
Briggs, Cooper & Torczon	1994	**Optimistic colouring.** Ancestor of most production allocators
George & Appel	1996	Iterated coalescing — removes redundant copy instructions
Poletto & Sarkar	1999	**Linear scan.** Drops graph colouring for speed; what JITs ship

The SSA line — and what it already established

Work	Year	Contribution
Tarjan & Yannakakis	1984	Linear-time chordality test and colouring order
Hack & Goos	2005–06	SSA interference graphs are **chordal** $\Rightarrow$ optimal colouring in linear time
Pereira & Palsberg	2005	Built a chordal allocator, measured 23,681 Java methods. $\sim$**95% of graphs are chordal**
Bouchez, Darte & Rastello	2006	The remaining hardness is in **spilling and coalescing**, not colouring

Literature Survey

Their measured results — we reproduce these, we don't discover them

Question	Published answer *(Pereira & Palsberg, Table 1)*
Optimal vs. classical heuristic?	**4.13 vs 4.17** registers/method — under 1%
Does φ-elimination destroy chordality?	**No — it improves it.** 91% $\rightarrow$ 95.5%
Does the advantage survive few registers?	Better at k=6, roughly equal at k=18

We are not claiming to discover any of this. These are the numbers we expect to reproduce — on a different processor, in our own compiler — and knowing them in advance is how we will tell whether our implementation is correct.

Motivation: Why Build Another One

The algorithms are known. The comparison between them is not reliable.

Every published comparison used a different compiler, target and benchmark set:

Work	Compiler	Target	Benchmarks
Pereira & Palsberg	JoeQ	Java bytecode	Java standard library
Hack & Goos	libFirm	x86	SPEC
Poletto & Sarkar	their own	x86	their own set
LLVM `greedy`	LLVM	many	LLVM's own suite

So the numbers can't be placed side by side. A difference between two published results might be the algorithm — or the compiler, the processor, or the test programs.

What we build instead

- **One** compiler, **one** IR, **one** benchmark set, **one** backend — with the allocator as a swappable part
- Every difference in output is then caused by the algorithm and nothing else
- On **RISC-V**, where this comparison hasn't been published

Problem Statement

Design and implement an SSA-based RISC-V compiler in which the register allocator is a swappable component, implement five allocation strategies, and compare them under identical conditions.

Three things we measure

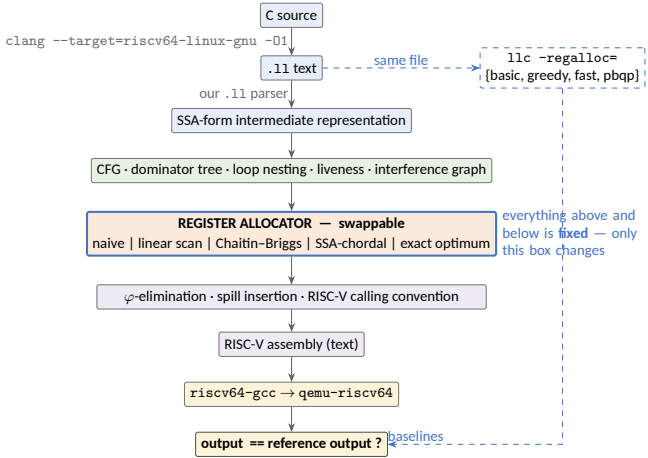
1. **Allocation quality.** How many spills does each algorithm produce on the same program? Compared against LLVM's production allocators, and against a true optimum from an exact solver on small functions.
2. **Effect of register count.** RISC-V has two official integer register configurations — **RV32I/RV64I with 32**, and **RV32E with 16**, used in embedded chips. We run every allocator on both. Real hardware, not an artificial setting.
3. **Cost of the algorithm.** How much work each allocator does, counted in operations rather than seconds, so the numbers reproduce on any machine.

Scope of Work

Layer	Components
Input	Textual LLVM IR (`.ll`) parser, defined subset
Analysis	CFG · dominator tree · loop detection · liveness · interference graph · chordality test
Allocation	**Five allocators:** naive · linear scan · Chaitin–Briggs · SSA/chordal · exact optimum · plus coalescing
Backend	φ-elimination · spill insertion · RISC-V calling convention · assembly emission
Validation	Static allocation verifier · differential testing · random program testing
Evaluation	Metrics · LLVM `llc` baselines · 16- vs 32-register comparison · plots · graph visualiser

Beating LLVM's `greedy` is not a goal. Landing within a constant factor and explaining the gap is.

Proposed Architecture



The Five Allocators

All five consume **the same interference graph** and **the same register budget**. Any difference in output is caused by the algorithm alone.

Allocator	What it does	Why it's in the study
Naive	Spill everything	Correctness floor. Gets the pipeline working before any real algorithm exists
Linear scan (1999)	One pass over live intervals, no graph	Speed baseline — what JITs ship
Chaitin–Briggs (1981/94)	Build → simplify → optimistic spill → select	The classical heuristic. Main comparison point
SSA / chordal (2005)	Colour along the order from Maximum Cardinality Search	**Optimal colouring in linear time**, because SSA graphs are chordal
Exact optimum	Integer linear program, off-the-shelf solver	Too slow for real use; small functions only, gives a **true reference point**
+ *Coalescing* (1996)	Removes redundant copies	Add-on to the graph-based allocators

Evaluation and Correctness

Metrics — all counted, never wall-clock

Metric	What it tells us
Static spill count	The headline quality number
Spills weighted by $10^{\text{loop depth}}$	Standard proxy for real runtime cost
Distance from the exact optimum	How far each heuristic really is from best possible
Executed instruction count (QEMU)	End-to-end quality, deterministic
Allocator operations executed	Cost side of the quality/cost trade

Register-count comparison — both are real RISC-V configurations, not an artificial knob

Profile	Integer registers
RV64I / RV32I	32
RV32E	**16** — official embedded profile, high pressure

Baselines: `llc -regalloc=`{basic, greedy, fast, pbqp} on **identical IR**

Correctness — an allocator that miscompiles silently invalidates every number. Three independent layers, starting week 1:

1. **Static verifier** — checks the allocation without running anything: no two simultaneously-alive values share a register, spill slots don't overlap
2. **Differential testing** — every benchmark's output compared against the `clang` build of the same source
3. **Random program generation** — auto-generated C programs, output compared against GCC

Work Plan and Current Status

Phase 1 — build the compiler (weeks 1–8)

Weeks	Work
1–2	Toolchain · end-to-end spike · `.ll` parser · IR structures · CFG
3–4	Dominator tree · loop detection · liveness · interference graph · naive allocator
5–6	Linear scan · Chaitin–Briggs
7–8	φ-elimination · RISC-V calling convention · verifier · differential testing

Phase 2 — the comparison (weeks 9–16)

Weeks	Work
9–10	Benchmarks · LLVM `llc` baselines · SSA/chordal allocator
11–12	Exact solver on small functions · coalescing
13–14	16- vs 32-register comparison across all allocators · slack
15–16	Results · visualiser · report

Current status

- Title and scope finalised with the guide
- Core literature reviewed, including the specific numbers we expect to reproduce
- Architecture and module interfaces specified; work divided across four tracks
- **Toolchain validated end-to-end.** C → `clang` → RISC-V IR → `llc` → assembly → `riscv64-linux-gnu-gcc` → `qemu-riscv64`, output identical to the host-compiled reference
- **All four LLVM baseline allocators confirmed on RISC-V — and they already differentiate.** On a small loop kernel, `fast` emits **20** stack accesses where `basic`, `greedy` and `pbqp` emit **2**

Applications

- **A controlled comparison that doesn't currently exist** — five allocators, one compiler, one benchmark set, one target, so the numbers are actually comparable
- **RISC-V embedded toolchains** — RV32E has only 16 registers. That's where allocator choice matters most, and it isn't well characterised
- **JIT compilers** — linear scan is deployed because colouring is assumed expensive. On SSA, optimal colouring is linear-time; our operation counts are the data needed to revisit that
- **Reusable artefacts** — a readable implementation of five allocators on a common IR (production allocators are effectively unreadable), a verification harness, and an interference-graph visualiser

Conclusion

- Register allocation is classically graph colouring, NP-hard in general. **For SSA it isn't** — optimal colouring is linear-time, established 2005
- The algorithms are known. **What's missing is a fair comparison** — every published result comes from a different compiler, target and benchmark set
- We build an SSA-based RISC-V compiler with a **swappable** allocator, implement five strategies, and compare them on both RISC-V register profiles
- **Next phase:** complete the analysis layer and classical allocators, reach a correct end-to-end pipeline, begin the comparison

Thank you — questions welcome.

References

1. G. J. Chaitin *et al.*, “Register allocation via coloring,” *Computer Languages*, vol. 6, no. 1, pp. 47–57, 1981.
2. P. Briggs, K. D. Cooper, and L. Torczon, “Improvements to graph coloring register allocation,” *ACM TOPLAS*, vol. 16, no. 3, pp. 428–455, 1994.
3. L. George and A. W. Appel, “Iterated register coalescing,” *ACM TOPLAS*, vol. 18, no. 3, pp. 300–324, 1996.
4. M. Poletto and V. Sarkar, “Linear scan register allocation,” *ACM TOPLAS*, vol. 21, no. 5, pp. 895–913, 1999.
5. S. Hack and G. Goos, “Optimal register allocation for SSA-form programs in polynomial time,” *Inf. Process. Lett.*, vol. 98, no. 4, pp. 150–155, 2006.
6. F. M. Q. Pereira and J. Palsberg, “Register allocation via coloring of chordal graphs,” in *Proc. APLAS*, 2005, pp. 315–329.
7. F. Bouchez, A. Darte, and F. Rastello, “Register allocation: What does the NP-completeness proof of Chaitin et al. really prove?,” in *Proc. LCPC*, 2006, pp. 283–298.
8. R. E. Tarjan and M. Yannakakis, “Simple linear-time algorithms to test chordality of graphs…,” *SIAM J. Comput.*, vol. 13, no. 3, pp. 566–579, 1984.
9. R. Castañeda Lozano and C. Schulte, “Survey on combinatorial register allocation and instruction scheduling,” *ACM Comput. Surv.*, vol. 52, no. 3, 2019.